

深圳大学实验报告

课程名称：移动互联网应用

实验项目名称：熟悉 Android 编程开发环境

学院：计算机与软件学院

专业：软件工程

指导教师：解为成

报告人：__学号：__班级：__

实验时间：2026 年 3 月 29 日

实验报告提交时间：2026 年 4 月 4 日

教务部制

实验目的与要求:

目的: 掌握面向 Android 编程的开发环境搭建。学习、掌握 Android 程序编写基本步骤, 例如, Android Studio 平台编写简单的一个 HelloWorld 程序, 掌握编译、运行等基本步骤和操作。

内容要求:

1. 搭建开发环境 (基本要求, 60 分):

(在自己电脑上) 下载安装和配置 Android Studio 开发环境, 并熟悉该开发环境中的常用操作, 并 New -> New Project -> Base Activity 一个项目, 而且能运行成功。请在报告中给出搭建环境成功的截图和简要文字说明, 简述自己所了解掌握的常用操作。

2. 新建的程序项目解析 (提升要求, 40 分)

通过查阅相关资料, 尝试对前面新建的 Base Activity 项目进行详细的解析, 包括代码的结构、每一部分的功能、每行代码表示的意义等。需要重点解析的代码见图 1。总体要求是解析越完整越好, 越详尽越好。

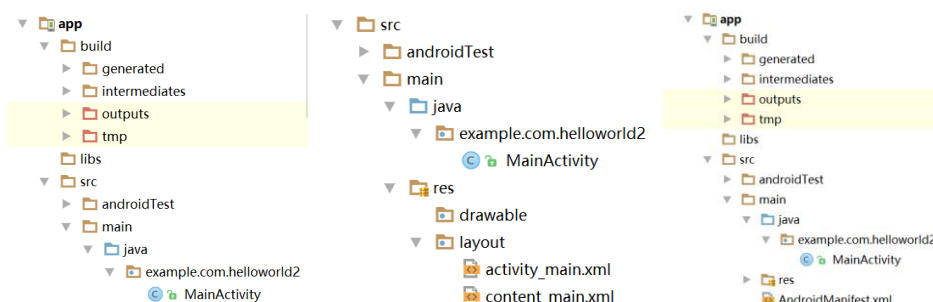


图 1 需要重点解析的代码, 主程序: MainActivity, 布局代码: activity_main.xml, content_main.xml, 注册代码: AndroidManifest.xml

注意:

1. 实验报告中需要有实验结果的截屏图像;
2. 尽量删除空白页。

方法、步骤:

1. 搭建开发环境（基本要求，60 分）:

（在自己电脑上）下载安装和配置 Android Studio 开发环境，并熟悉该开发环境中的常用操作，并 New -> New Project ->Base Activity 一个项目，而且能运行成功。在报告中给出搭建环境成功的截图和简要文字说明，简述所了解掌握的常用操作。

2. 新建的程序项目解析（提升要求，40 分）

通过查阅相关资料，尝试对前面新建的 Base Activity 项目进行详细的解析，包括代码的结构、每一部分的功能、每行代码表示的意义等。

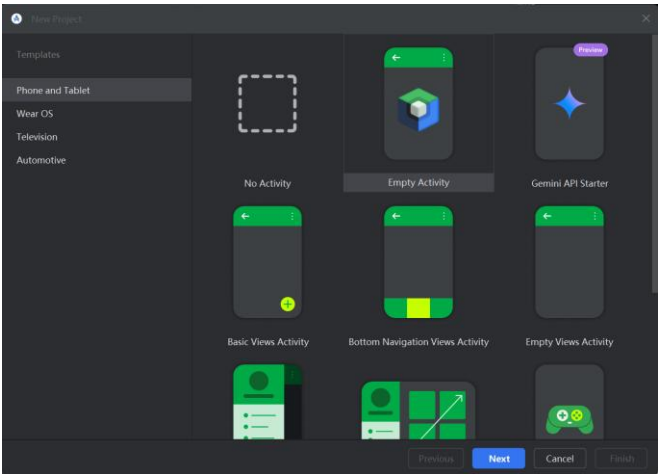
实验过程及内容:

1. 搭建开发环境（基本要求，60 分）:

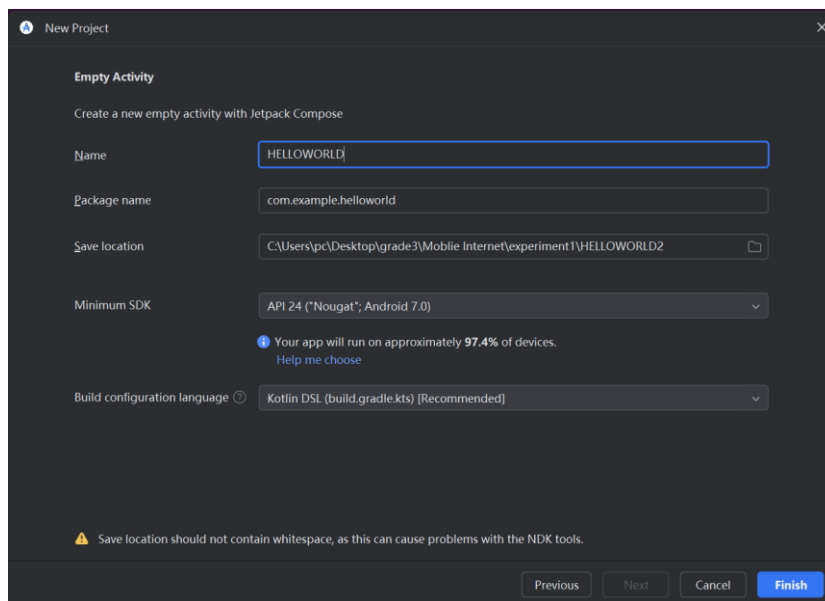
（1）下载 Android Studio 并安装

平台	Android Studio 软件包	大小	SHA-256 校验和
Windows (64 位)	android-studio-2024.2.2.13-windows.exe 推荐	1.2 GB	7d93dd9bf3539f948f609b1968507b1f502bf6965d2d44bd38a17ff26cb5dd3e

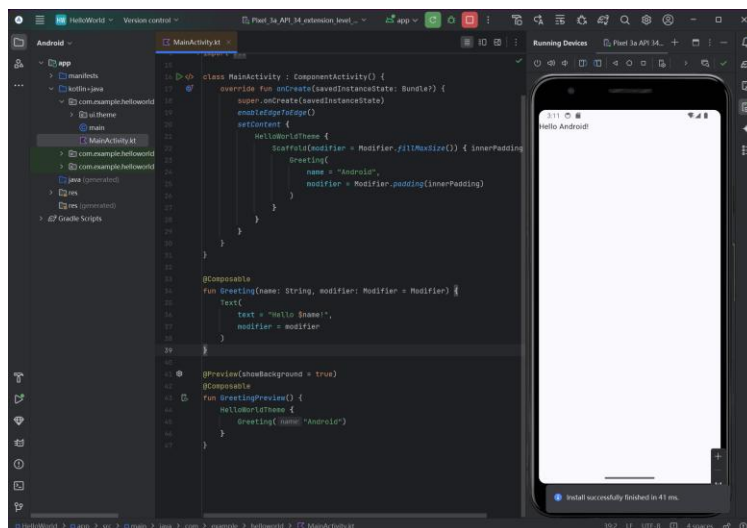
（2）打开代码编辑器后，点击左上角 File → New 进入新建项目界面



(3) 输入新项目名 **hello world** 并选择 SDK 版本为 **API 24**, 使用 **kotlin** 构建代码



(4) 等待项目自动搭建并成功运行



2. 新建的程序项目解析（提升要求，40 分）

(1) 功能解释:

代码的功能是, 显示一个问候语“Hello Android!”。

(2) 代码的详细解释:

2.1 MainActivity 类

Activity 类是安卓页面的核心组件, 每个页面对应一个 activity, 而 MainActivity 是程序运行时首先展示的页面

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            HelloWorldTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

```

- **MainActivity** : 继承自 `ComponentActivity`, 是 Android 中创建应用的基础组件。
- **onCreate** : `onXXX` 的表达类似于 QT 中信号的表示, 表示创建时调用
- **enableEdgeToEdge()** : 使得应用界面从屏幕的边缘延伸到边缘
- **setContent** 设置 Compose UI 内容。
- **HelloWorldTheme** 是定义的主题包装器, 应用主题样式。

2.2) Greeting 函数

```

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

```

- **@Composable** 表示这是一个 Composable 函数, 它可以在 Compose UI 中创建可组合的 UI 元素。
- **Greeting** 参数: `name` (显示的文字) 和 `modifier` (用于修饰该 UI 元素的 `Modifier`)。
- `Text` 组件用于显示文本内容, 这里显示的是“Hello \$name!” (“Hello Android!”), 其中 `name` 是传入的参数。

2.3) GreetingPreview 函数

```

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

```

- **@Composable** 表示这是一个 Composable 函数, 它可以在 Compose UI 中创建

可组合的 UI 元素。

- Greeting 参数: name (显示的文字) 和 modifier (用于修饰该 UI 元素的 Modifier)。
- Text 组件用于显示文本内容, 这里显示的是“Hello \$name!” (“Hello Android!”), 其中 name 是传入的参数。

2.4) GreetingPreview 函数

```
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    HelloWorldTheme {
        Greeting(name: "Android")
    }
}
```

- @Preview 使得可以在 Android Studio 中预览 Composable 函数。在这段代码中, GreetingPreview 函数会显示 Greeting 组件的预览, 显示“Hello Android!”。
- showBackground = true 表示预览中将显示一个背景色。
- HelloWorldTheme 是应用的主题, 包装在预览中, 保持一致的样式。
-

(3) 项目结构解析:

3.1) kotlin+java/ui/theme

其中存放

Color.kt 定义单个颜色有关的变量

Theme.kt 定义 UI 界面的

Type.kt 字体文件

3.2) MainActivity.kt

程序入口, 创建应用的基础组件。

res/: 存放所有资源文件, 如布局、图片、字符串等。

drawable/: 存放图形布局文件

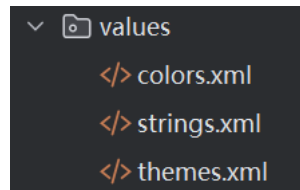
```
▼ drawable
  </> ic_launcher_background.xml
  </> ic_launcher_foreground.xml
```

这两个文件分别是程序加载时开屏界面的背景和前景

mipmap/: 存放应用的启动图标, 其中图标按分辨率不同分为不同 mipmap, 使得图标在缩放时能通过 mipmap 图像放大 / 缩小。

```
▼ mipmap
  ▼ ic_launcher (6)
    ic_launcher.webp (hdpi)
    ic_launcher.webp (mdpi)
    ic_launcher.webp (xhdpi)
    ic_launcher.webp (xxhdpi)
    ic_launcher.webp (xxxhdpi)
    </> ic_launcher.xml (anydpi-v26)
```

values/: 存放各种资源，如字符串、颜色和样式。



例如 strings.xml 中存放了 app 名称

```
<resources>
    <string name="app_name">HelloWorld</string>
</resources>
```

xml/: 存放 XML 布局文件

3.3) AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
```

xmlns:android 和 xmlns:tools，用于定义 Android 属性和工具属性。

```
<application
    android:allowBackup="true"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.HelloWorld"
    tools:targetApi="31">
```

android:allowBackup="true": 允许应用的数据进行备份。

android:dataExtractionRules="@xml/data_extraction_rules": 指示应用数据的提取规则

android:fullBackupContent="@xml/backup_rules": 定义应用全量备份的内容。

android:icon="@mipmap/ic_launcher": 指定应用的图标资源

android:label="@string/app_name": 应用的标签

android:roundIcon="@mipmap/ic_launcher_round": 指定应用的圆形图标

android:supportsRtl="true": 指示应用支持从右到左语言

android:theme="@style/Theme.HelloWorld": 定义了主题样式

tools:targetApi="31": API 版本

```
    android:name=".MainActivity"
    android:exported="true"
    android:label="@string/app_name"
    android:theme="@style/Theme.HelloWorld">
```

`android:name=".MainActivity"`: 指定了应用的主活动类 `MainActivity`
`android:exported="true"`: 表示该活动是可以被其他应用或系统调用的
`android:label="@string/app_name"`: 为应用指定标签名称, 通常显示在设备的 UI 中。
`android:theme="@style/Theme.HelloWorld"`: 主题样式。

(4) UI 组件添加

4.1) Button 按钮和 Toast 使用

4.1.1) 介绍:

在 Android 系统中, `Button` 和 `Toast` 是两个常用的 UI 元素, 分别用于用户交互和显示临时消息。

Button

`Button` 是一个用于接收用户点击的 UI 元素。通常在布局文件中定义, 并通过点击事件来触发某些操作。

Toast

`Toast` 是一个用于显示简短消息的 UI 元素, 它不会干扰用户的操作, 显示时间短, 通常用于提示用户一些信息。

4.1.2) 在 XML 布局文件中定义 Button 元素:

```
<Button
    android:id="@+id/startUpButton"
    android:layout_width="100dp"
    android:layout_height="100dp"
    android:layout_marginStart="24dp"
    android:layout_marginTop="32dp"
    android:clickable="true"
    android:focusable="true"
    android:linksClickable="false"
    android:longClickable="false"
    android:onClick="startUp"
    android:text="Start"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.0" />
```

参数说明

1. `android:id`

用来定义该按钮的 ID, 可以在 Java 或 Kotlin 代码中通过 `findViewById` 方法引用。

2. `android:layout_width` 和 `android:layout_height`

表示按钮的宽度和高度。设置按钮的大小, 确保在不同设备上有一致的显示效果。

3. `android:layout_marginStart` 和 `android:layout_marginTop`

用来设置按钮与父布局 (或者其他元素) 之间的距离。

4. `android:clickable` 和 `android:focusable`

表示按钮可以被点击和聚焦。

5. `android:linksClickable` 和 `android:longClickable`

表示按钮不支持点击链接和长按事件。

6. `android:onClick`

`startUp` 是一个方法名，表示当用户点击按钮时，执行 `startUp` 方法。

7. `android:text`

`Start` 是按钮上显示的文本。

8.

`layout_constraintBottom_toBottomOf="parent"`: 将按钮的底部与父布局的底部对齐。

`layout_constraintEnd_toEndOf="parent"`: 将按钮的右边与父布局的右边对齐。

`layout_constraintStart_toStartOf="parent"`: 将按钮的左边与父布局的左边对齐。

`layout_constraintTop_toTopOf="parent"`: 将按钮的顶部与父布局的顶部对齐。

4.1.3) 在 java 代码中使用 Toast 在 Button 被点击时弹出 toast 提示

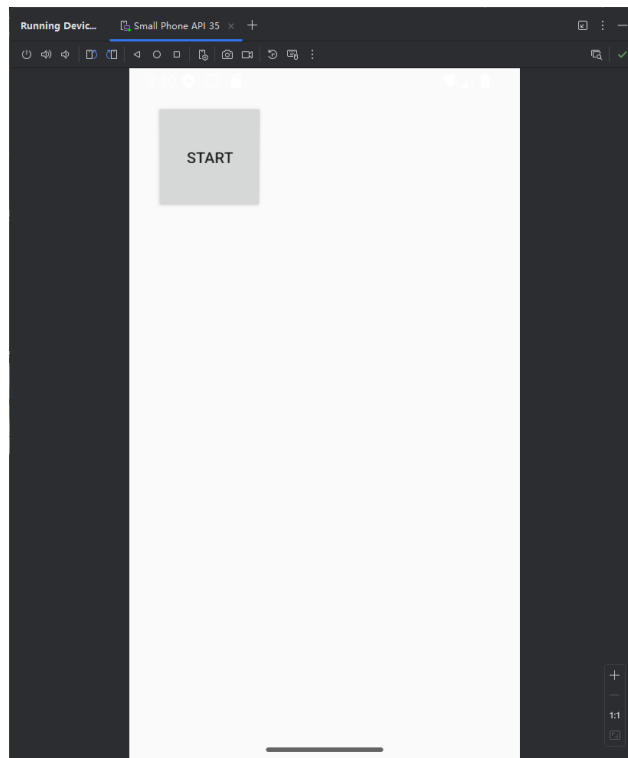
解释: `setContentView` 设定 MainActivity 布局, `setOnClickListener` 设定了当按钮被点击时触发的事件, 产生一个 Toast 并显示 Hello World 提示

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

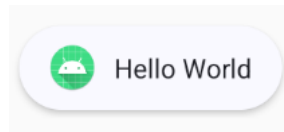
        Button startUpButton = findViewById(R.id.startUpButton);
        startUpButton.setOnClickListener(v -> {
            Toast.makeText(context MainActivity.this, text: "Hello World", Toast.LENGTH_SHORT).show();
        });
    }
}
```

4.1.4) 运行结果

Button 示例



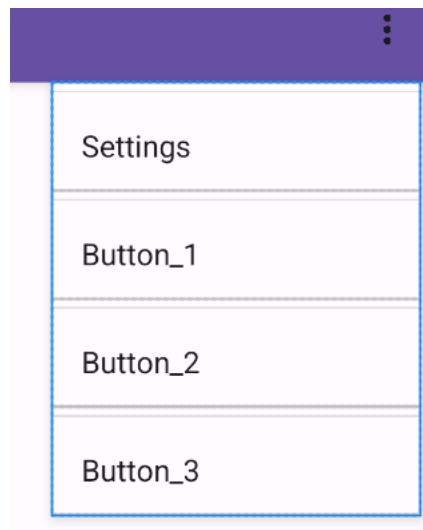
当 Button 被点击时
弹出 Hello World 提示框



4.2) Menu 的使用

4.2.1) 添加 menu 布局文件

在 res/menu 文件夹中创建 menu_main.xml 并添加四个 item
此时布局界面显示



4.2.2) 添加按钮被点击后触发逻辑

在 MainActivity.java 文件中添加每个按钮被点击时的逻辑:

```
//noinspection SimplifiableIfStatement
if (id == R.id.action_settings) {
    Toast.makeText(context, this, text: "Button Setting Clicked", Toast.LENGTH_LONG).show();
    return true;
}

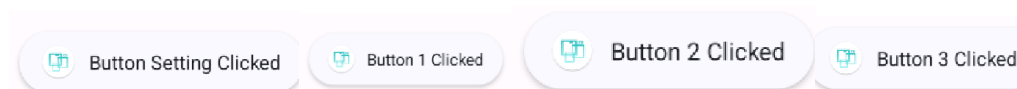
if (id == R.id.Button_1) {
    Toast.makeText(context, this, text: "Button 1 Clicked", Toast.LENGTH_SHORT).show();
    return true;
}

if (id == R.id.Button_2) {
    Toast.makeText(context, this, text: "Button 2 Clicked", Toast.LENGTH_SHORT).show();
    return true;
}

if (id == R.id.Button_3) {
    Toast.makeText(context, this, text: "Button 3 Clicked", Toast.LENGTH_SHORT).show();
    return true;
}
```

四个选项被选择时分别会有一个 Toast 显示当前按钮被点击

4.2.3) 运行



数据处理分析：

- 1.成功在 XML 布局中定义 Button 元素并执行 setContent 操作
- 2.成功布局并使用 Menu

实验结论：

本次实验成功在个人电脑上安装并配置了 **Android Studio** 开发环境。创建了一个新项目，并成功运行了基础的 **HelloWorld** 程序。配置 Gradle 构建脚本，管理项目依赖。

了解了安卓项目结构，其关键部分为：

MainActivity：这是主活动类，用于展示应用的启动界面，继承自 **ComponentActivity**，设置了布局并执行了 **setContent** 操作。

AndroidManifest.xml：声明应用的组件、权限以及主题。此文件配置了应用的启动活动 **MainActivity**，并设置了应用的图标、名称等。

通过这次实验，成功搭建了 Android 开发环境，熟悉了 Android Studio 的基本操作，并理解了简单 Android 应用项目的结构和功能，通过添加 Button, Menu 和使用 Toast 理解了一些安卓组件的使用方法。

指导教师批阅意见：

成绩评定：

指导教师签字：解为成
2026 年 3 月 日

备注：

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。